Ripple-Down Rules (RDR)

An Alternative to Machine Learning

Dr. Ye Kyaw Thu,
LU Lab., Myanmar

12 April 2026

Table of Contents

1. Introduction
2. Knowledge Acquisition
3. RDR Overview
4. Single Classification RDR
5. Multiple Classification RDR
6. General RDR
7. Deployment
8. ML vs RDR
9. Applications
10. Summary
11. References

The Knowledge Acquisition Bottleneck

- **The Challenge:** Traditional AI (Expert Systems) suffered from the "bottleneck" where capturing general knowledge from experts was slow and error-prone (Feigenbaum, 1984).
- **Expertise Paradox:** Experts are often excellent at solving specific cases but poor at articulating the general rules they use.
- **RDR Philosophy:** Instead of asking for general rules, RDR captures knowledge incrementally as "justifications" for specific case classifications.

Situated Knowledge and Rationality

- **Situated Knowledge:** Knowledge is not a standalone object; it is always tied to the context of a specific case and the current state of the system.
- **Principle of Rationality:** If an expert disagrees with a system's conclusion, they must provide a "differentiating feature" that distinguishes the current case from previous ones (Compton & Kang, 2021).
- **Incremental Growth:** Systems are built while in use, ensuring the knowledge base (KB) is always grounded in real-world data.

Core Principles of RDR

- **Case-Driven:** Every rule is created because the system failed on a specific case.
- **Cornerstone Cases:** The specific case that triggered a rule's creation is stored as a "Cornerstone Case" to prevent future regressions.
- **No Retraction:** Once a rule is added, it is never deleted or edited; it is only "refined" by child rules that handle more specific scenarios.

Single Classification RDR (SCRDR)

- **Structure:** A binary tree where each node is a rule.
- **Evaluation:**
  - If a rule's condition is **True**, the system tries its "True" child (refinement).
  - If **False**, it tries its "False" sibling (next alternative).
- **Result:** The conclusion of the last rule to evaluate as "True" in the path is the final output.
- Replacement logic.

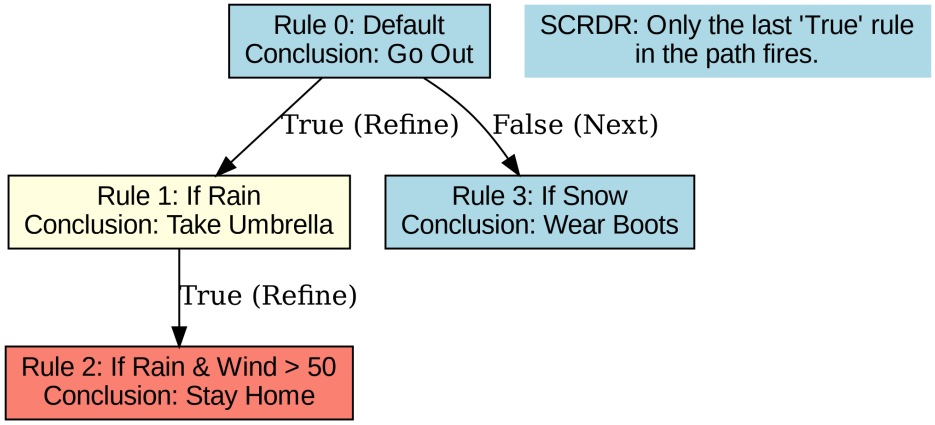

Figure: SCRDR Binary Tree Path

Adding Rules in SCRDR

- **Refinement (True Child):** Added when the system gives a conclusion, but the expert says it is wrong. The new rule corrects the parent's over-generalization.
- **Alternative (False Sibling):** Added when the system gives no conclusion (or a default conclusion), but the expert wants to assign one.
- **Verification:** The new rule must distinguish the current case from the parent's cornerstone case to ensure logical consistency.

SCRDR Convergence and Limitations

- **Convergence:** Math models show SCRDR KBs eventually stabilize and reach high accuracy regardless of the expert's initial rule quality.
- **The Repetition Problem:** A major downside of SCRDR is that the same rule might need to be added in multiple branches if the tree structure becomes overly specific to one path.
- KB = Knowledge Base (The set of rules and cornerstone cases).
- In RDR: The KB is the *actual tree structure (SCRDR/MCRDR)* that grew from the expert's justifications.

Multiple Classification RDR (MCRDR)

- **Transition:** MCRDR was developed to allow multiple independent conclusions for a single case (e.g., a patient having both "Flu" and "Anemia").
- **N-ary Tree:** Instead of a binary tree, MCRDR uses a structure where all siblings at the same level are evaluated.
- **Refinement:** A rule only refines its direct parent, not all preceding rules in the path.
- Parallel logic.

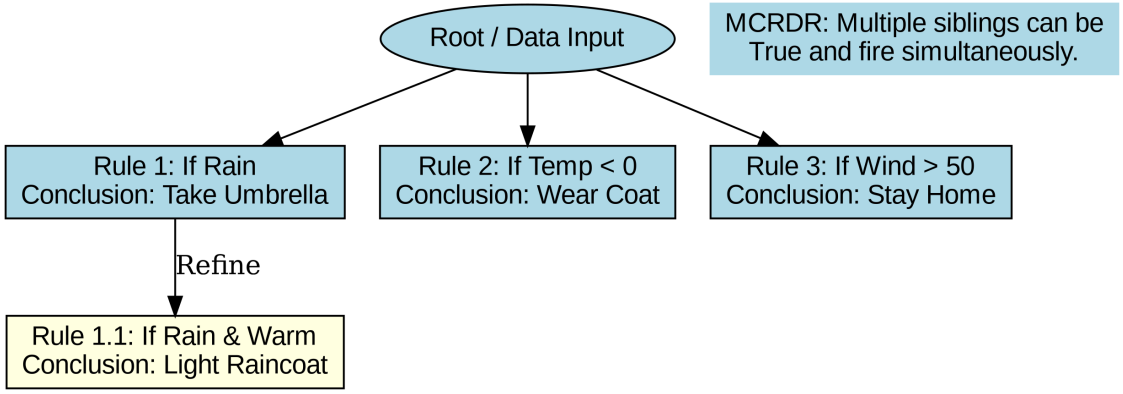

Figure: MCRDR N-ary Structure

MCRDR Inference Logic

- The system evaluates all rules at the top level.
- For every rule that is **True**, the system then evaluates all its children.
- This continues until no more refinements fire. Every "terminal" True rule in a path contributes its conclusion to the final set.

Knowledge Maintenance in MCRDR

- **Modularization:** Knowledge is more modular than in SCRDR, as different types of conclusions (e.g., diagnosis vs. treatment) can reside in different branches.
- **Efficiency:** MCRDR significantly reduces rule repetition because independent concepts do not need to be nested within each other.

MCRDR: User Experience

- **Speed:** Experts can add rules very quickly (often < 1 minute) because they only focus on justifying the specific new conclusion.
- **Validation:** The system automatically checks the new rule against all cornerstone cases that reached that node, ensuring no old cases are misclassified.

General RDR (GRDR) and Chaining

- **The Goal:** To achieve the power of general production systems (like CLIPS or OPS5) while maintaining RDR's ease of maintenance.
- **Inference Cycles:** GRDR supports looping where conclusions from one rule can trigger other rules in a subsequent cycle.
- **Monotonicity:** Once a fact is asserted, it remains true for that case, preventing complex "truth maintenance" issues common in standard rule-based systems.

Heuristic Classification in GRDR

- GRDR allows for "Data Abstraction" where raw data is converted into intermediate concepts (e.g., '$temp > 38^{\circ}$ ' $\rightarrow$ 'high_fever').
- These intermediate concepts are then used by higher-level rules to reach final classifications.
- Reasoningchaining logic.

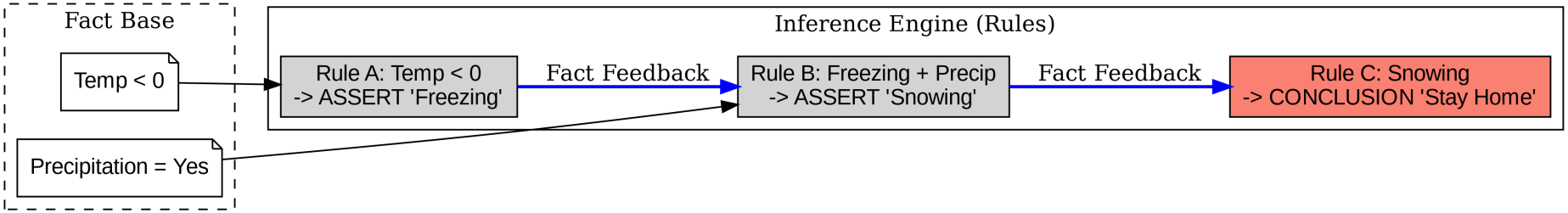

Figure: GRDR Chaining Process

GRDR vs. Conventional Production Rules

- **Standard Rules:** Adding a new rule can have unpredictable side effects on the entire system's logic.
- **GRDR:** Every new rule is added as a refinement of a specific existing rule, localizing the change and making the impact predictable and verifiable.

Incremental Implementation Strategy

- **Run-time Development:** RDR systems are typically deployed with an empty or minimal KB.
- **Expert-in-the-loop:** When the system encounters a case it can't handle, the expert is prompted to add a rule "on the fly."
- **Safety:** The system can flag "low confidence" if no rules fire or if a new case is significantly different from all cornerstones.

Data Abstraction and Interfaces

- To make RDR effective, raw data must be abstracted into a format experts can use for conditions.
- **Abstraction Layer:** Translates database fields or sensor logs into meaningful linguistic terms (e.g., "Patient is tachycardic"). *Tachycardic definition: A heart rate over 100 beats per minute (BPM) at rest.* ဂရိစကားက လာတဲ့ ဆေးပညာဝေါဟာရပါ တာကီ ဆိုတာက မြန်နှုန်း၊ ကာဒီယာ ဆိုတာက နှလုံး ဆိုတဲ့ အဓိပ္ပါယ်။
- **UI Design:** The interface must allow experts to easily browse case history and select differentiating features.

Data Requirements and Labels

- **Machine Learning (ML):** Requires thousands of labeled examples (Big Data) to achieve high accuracy.
- **RDR:** Requires **zero** initial data. It learns from a single instance as soon as an expert provides a justification.
- **Labeling Effort:** In ML, labeling is a separate, often tedious task. In RDR, labeling is the knowledge acquisition process.

Handling Rare Cases (The Long Tail)

- **ML Problem:** Statistical models often ignore "outliers" or rare classes because they don't appear enough in the training set.
- **RDR Solution:** An expert can add a rule for a "one-in-a-million" case as soon as they see it. RDR excels at handling the "long tail" of expert domains.

Explainability and Transparency

- **Black Box:** Deep Learning models can be accurate but cannot explain "why" they made a choice.
- **White Box:** RDR is inherently explainable. The explanation for a conclusion is the path of rules that fired, expressed in the expert's own terms.

The Maintenance Cost

- **ML Drift:** When data distributions change, ML models must be retrained on new datasets, which is costly and time-consuming.
- **RDR Adaptability:** Experts simply add new refinement rules as soon as they notice the environment has changed, keeping the system up-to-date in real-time.

Comparison Summary Table

Feature	**Machine Learning**	**Ripple-Down Rules**
Data Needed	Large Volume	Single Cases
Knowledge Source	Data (Labels)	Human Expert
Logic	Statistical	Symbolic/Contextual
Maintenance	Retraining	Incremental Addition
Explainability	Low (Black Box)	High (Natural Lang)

Industrial Success: Pathology and Beyond

- **PEIRS (Pathology Expert Interpretative Reporting System) & PKS (Pacific Knowledge Systems):** Clinical systems (e.g., LabWizard) that have been used to generate millions of medical reports worldwide.
- **Address Cleaning:** IBM used RDR to handle the massive variety and noise in Indian street address data (unstructured text).
- **Social Media:** Tapacross used RDR for sentiment analysis and trend prediction where context changes rapidly.

Research-Demonstrated Applications

- **NLP:** Information extraction and part-of-speech tagging for low-resource languages.
- **Robotics:** Used to teach robots complex tasks through demonstration and incremental correction.
- **Image Processing:** Parameter selection and anomaly detection in video streams.

Conclusion

- RDR provides a pragmatic way to build expert systems without the "knowledge engineering" overhead.
- It is particularly powerful in domains where data is scarce but human expertise is available.
- By treating knowledge as *situated* and *incremental*, RDR avoids the brittleness of traditional AI and the data-hunger of ML.

Conclusion

- Ripple-Down Rules: *The Alternative to Machine Learning* by Paul Compton & Byeong Ho Kang စာအုပ်ကို အခြေခံပြီး ဒီ presentation slide ကို ပြင်ခဲ့တာပါ။
- အဲဒီစာအုပ်ထဲမှာ ဆရာတို့ Language Understand Lab. က ဆရာ့တပည့်တွေနဲ့ အတူ ရေးထားတဲ့ စာတမ်းနှစ်စောင်အကြောင်းကို reference အနေနဲ့ထည့်ထားတာမို့လို့ ဝမ်းသာရပါတယ်။

Conclusion

A2.2.9 Other Language Tasks

To facilitate string similarity measurement RDR has been used for processing graphemes and phonemes for the Burmese language (Wai et al.).

Appendix 2 ■ **151**

They have also been used to produce consistent Latin script versions of Burmese proper names (Zaw et al. 2020).

Figure: Appendix 2 of the book: *Ripple-Down Rules: The Alternative to Machine Learning* by Paul Compton & Byeong Ho Kang, page 150 and 151

Conclusion

- Khaing Hsu Wai, Ye Kyaw Thu, Swe Zin Moe, Hnin Aye Thant, Thepchai Supnithi, "Myanmar (Burmese) String Similarity Measures based on Phoneme Similarity", Journal of Intelligent Informatics and Smart Technology, April 1st Issue, 2020, pp. 27-34.
- Wint Theingi Zaw, Shwe Sin Moe, Ye Kyaw Thu, Nyein Nyein Oo, "Applying Weighted Finite State Transducers and Ripple Down Rules for Myanmar Name Romanization", In Proceedings of the 17th International Conference on Electrical Engineering/Electronics, Computer, Telecommunications and Information Technology (ECTI-CON 2020), June 24-27, 2020, Virtual Conference Hosted by College of Computing, Prince of Songkla University, Thailand, pp. 143-148.
- တကယ်က RDR ကိုသုံးထားတဲ့ စာတမ်းက အဲဒီနှစ်စောင်ထဲမဟုတ်ပါဘူး။ ဆရာ့ homepage: `https://sites.google.com/site/yekyawthunlp/` မှာ ရှာကြည့်ကြပါ။ :)

Thank You!

Questions?

References

- Compton, P., & Kang, B. H. (2021). *Ripple-Down Rules: The Alternative to Machine Learning*. CRC Press.
- Compton, P., Edwards, G., Srinivasan, A., et al. (1992). *Ripple down rules: possibilities and limitations*. Proceedings of the 6th Banff Knowledge Acquisition Workshop.
- Feigenbaum, E. A. (1984). *Knowledge Engineering: The Applied Side of Artificial Intelligence*. Annals of the New York Academy of Sciences.
- Kang, B. H., Compton, P., & Preston, P. (1995). *Multiple Classification Ripple Down Rules*. Proceedings of the 9th Banff Knowledge Acquisition Workshop.
- `https://en.wikipedia.org/wiki/Ripple-down_rules`